Lesson 13：HTTP、JSON 与本地服务器

LLM Learn 大模型开发课程

2026 年 11 月

目录

1 教学目标与先修 2

2 HTTP 请求与响应 2

3 JSON 编码与字节数 2

4 服务端代码结构 2

5 错误响应与安全限制 2

6 核心关系或公式 2

7 本课完整源码：逐行中文注释 2

8 练习与验收 4

9 执行记录与参考 4

1 教学目标与先修

掌握请求方法、状态码、Headers/Body、UTF-8 和基本本地服务器的实现。

先修：Python 类、字典、异常与文件处理。预计 90–120 分钟。

2 HTTP 请求与响应

请求由 method、URL/路径、请求头和可选请求体构成；响应含 status、headers 和 body。API 是可调用的契约，HTTP 是实现协议之一。

3 JSON 编码与字节数

json.dumps 将 dict 编成字符串, encode('utf-8') 才生成通过 HTTP 发送的字节。Content-Length 必须是字节长度，不是 Unicode 字符数。

4 服务端代码结构

DemoHandler 继承 BaseHTTPRequestHandler；send_json 负责发送响应头和正文；do_GET 处理 /health；do_POST 验证 /echo、Content-Type、长度和 JSON 结构；main 绑定 127.0.0.1:8765。

5 错误响应与安全限制

/health 返回 200；未知路径为 404；非 JSON 为 415；无效请求为 400。该服务只用于本地学习，缺少生产级认证和限速；切勿暴露在公网。

6 核心关系或公式

$$N_{\text{bytes}} = |\text{UTF8}(\text{JSON}(D))|$$

7 本课完整源码：逐行中文注释

下列程序与本课 code 目录的实际源文件完全对应。每个模块、函数及关键可执行语句都配有中文说明。

```
"""Lesson 13: 仅绑定 127.0.0.1 的教学 HTTP/JSON 服务；不是生产级服务器。"""

# 导入 json，负责 JSON 文本编码与解码。
import json
# 导入标准库的 HTTP 服务器基础类和多线程服务器。
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
```

```
class DemoHandler(BaseHTTPRequestHandler):
    """ 处理 /health、/echo 与错误路径，统一返回 JSON。"""

    def send_json(self, status_code: int, payload: dict) -> None:
        """ 发送 JSON 响应；status_code 为 HTTP 状态码，payload 为字典。"""
        # 把 Python 字典序列化成 UTF-8 字节，并保留中文字符。
        body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
        # 向客户端发送 HTTP 状态行，例如 200 OK。
        self.send_response(status_code)
        # 明确告知客户端返回的是 UTF-8 编码的 JSON。
        self.send_header("Content-Type", "application/json; charset=utf-8")
        # 告知正文的字节数，便于客户端正确读取。
        self.send_header("Content-Length", str(len(body)))
        # 结束所有响应头部，准备发送响应体。
        self.end_headers()
        # 通过网络输出流写入 JSON 正文。
        self.wfile.write(body)

    def do_GET(self) -> None:
        """ 处理 HTTP GET 请求；只开放健康检查路由。"""
        # 精确比较请求路径，防止意外开放其他资源。
        if self.path == "/health":
            # 成功时返回 JSON 与 200 状态码。
            self.send_json(200, {"status": "ok", "message": " 本地 API 运行正常"})
            # 提前返回，避免进入后续错误处理。
            return
        # 未定义路由明确返回 404，不允许静默成功。
        self.send_json(404, {"error": "route_not_found"})

    def do_POST(self) -> None:
        """ 处理 /echo 的 JSON 正文，并拒绝错误内容或超长请求。"""
        # 仅允许向 /echo 路由提交数据。
        if self.path != "/echo":
            # 路由不存在时返回 404。
            self.send_json(404, {"error": "route_not_found"})
            # 退出本次请求处理。
            return
        # 读取 Content-Type，防止把普通字符串误当成 JSON。
        content_type = self.headers.get("Content-Type", "")
        # 检查请求体的媒体类型是否是 application/json。
        if not content_type.startswith("application/json"):
            # 返回 415，表示不支持的请求体类型。
            self.send_json(415, {"error": "expected_json"})
            # 退出本次请求处理。
            return
        try:
            # 解析客户端声明的正文长度（单位为字节）。
            length = int(self.headers.get("Content-Length", "0"))
            # 对请求体增加上限，防止教学服务器读入过量数据。
            if not 0 < length <= 10000:
                # 400 表示正文长度不符合要求。
                self.send_json(400, {"error": "invalid_length"})
                # 直接结束，不继续读写网络数据。
                return
            # 从网络读取指定字节数并解析 JSON 对象。
            data = json.loads(self.rfile.read(length))
```

```
        except (ValueError, UnicodeDecodeError):
            # 无法解析数字或 JSON 时返回明确错误。
            self.send_json(400, {"error": "invalid_json"})
            # 避免在错误后继续使用未定义 data。
            return
        # 检查数据是否是字典，并要求 text 字段必须是字符串。
        if not isinstance(data, dict) or not isinstance(data.get("text"), str):
            # 错误的 JSON 结构同样属于不合格请求。
            self.send_json(400, {"error": "text_must_be_string"})
            # 结束错误分支。
            return
        # 成功返回收到的文本及 Unicode 字符数量（非字节数）。
        self.send_json(200, {"received": data["text"], "length": len(data["text"])})


def main() -> None:
    """ 启动绑定本机 8765 端口的服务，Ctrl+C 后安全关闭。"""
    # 绑定 127.0.0.1，避免教学服务器默认暴露在局域网。
    server = ThreadingHTTPServer(("127.0.0.1", 8765), DemoHandler)
    # 告知用户访问地址以及停止服务的方式。
    print(" 服务地址：http://127.0.0.1:8765，按 Ctrl+C 停止")
    try:
        # 持续接受 HTTP 请求，直到用户中断程序。
        server.serve_forever()
    except KeyboardInterrupt:
        # 用户按 Ctrl+C 时输出结束信息。
        print("\n 已停止服务")
    finally:
        # 无论正常结束还是抛错，都释放监听套接字。
        server.server_close()


if __name__ == "__main__":
    # 只在直接运行时启动网络服务。
    main()
```

8 练习与验收

- 启动服务器后用 curl 请求 /health。
- 解释 len(body) 为什么与 len(中文字符串) 可能不同。
- 本地真实套接字测试正常 GET 与 404。

9 执行记录与参考

使用 python -m unittest discover -s tests -v 执行测试。请以本课 results/verification.md 中的真实结果和 GitHub Actions 日志为准，Mock 并非在线 API 验证。

- https://git-scm.com/docs
- https://docs.github.com/en/actions

- https://developer.mozilla.org/en-US/docs/Web/HTTP
- https://platform.openai.com/docs/api-reference/responses